

Evaluation of RTOS for Robotic Applications with ROS 2 on Embedded Systems

Jin-Ho Kim¹, Jun-Hyeon Choi¹, Ye-Chan An¹ and Tae-Yong Kuc^{2*}

¹Department of Electrical and Computer Engineering, Sungkyunkwan University,
Suwon, 16419, Korea ({jinho9812, choijunhyeon, ycan}@g.skku.edu)

²Department of Electrical and Computer Engineering, Sungkyunkwan University,
Suwon, 16419, Korea (tykuc@skku.edu) * Corresponding author

Abstract: This paper aims to establish a quantitative foundation for applying real time operating systems to robotics systems, where general purpose operating systems like Linux are commonly used but often fail to meet strict timing requirements, especially in time critical domains such as autonomous vehicles, missile guidance, surgical robotics, and nuclear systems. Although ROS2 has become the standard middleware in modern robotics, its deployment on real time operating systems remains rare due to lack of empirical evidence and practical adoption experience in the robotics domain. To address this gap, we quantitatively benchmark ROS2 communication latency across three platforms: standard Linux, soft real time Linux with a PREEMPT RT patch, and the commercial RTOS QNX. Experiments are conducted on an embedded board using identical publisher subscriber logic, under varying message sizes, publication frequencies, intra and inter process configurations, and stress conditions. The results show that QNX consistently achieves lower and more stable latency for small high frequency messages, but suffers degradation under multi node stress scenarios. Based on these findings, we propose a layered control structure in which RTOS handles low level real time tasks such as motor control and sensor feedback, while high level planning and computation remain on GPOS. This study provides a practical reference for robotics system designers considering real time architectures and outlines the trade offs and deployment strategies for adopting RTOS in embedded robotic control.

Keywords: ROS2, Real-Time Systems, Layered Control Architecture, End-to-End Latency, QNX, Linux

1. INTRODUCTION

In recent years, most robotic systems have been developed on general-purpose operating systems (GPOS), such as Ubuntu Linux, utilizing the Robot Operating System 2 (ROS 2) as their primary middleware. While such platforms are sufficient for general application development and non-real-time processing, they reveal significant limitations in mission and time critical applications[1] such as aerospace, autonomous vehicles, and medical robotics.

Robotic systems typically consist of multiple concurrently running nodes that exchange high-frequency and large-volume data. Under such heavy workloads, GPOS platforms like Linux based on the Completely Fair Scheduler (CFS)[2] or round-robin scheduling struggle to guarantee deterministic execution timing. This unpredictability makes them unsuitable for systems that require strict real-time responsiveness and guaranteed timing behavior.

To address this issue, real-time operating systems (RTOS)[3] have become increasingly important. RTOS provide preemptive scheduling based on task priorities, enabling deterministic task execution. Although some systems adopt the Linux RT patch to enhance real-time performance, higher levels of determinism and reliability often require full-fledged RTOS such as QNX or Vx-Works.

Among these, QNX is a well-established RTOS widely used in automotive and safety-critical domains. Its microkernel architecture[4] offers several advantages:

fast context switching, the absence of demand paging(eliminating page faults), and a minimal kernel footprint running most system services in user space. These feature enable QNX(RTOS) to deliver both high performance and system stability, making it highly suitable for precise embedded control applications.

This study was to quantitatively evaluate the impact of RTOS adoption in robotic systems, specifically by benchmarking end-to-end latency[5] of ROS 2 communication under three mentioned os(standard Linux, Linux with RT patch, and QNX). Using identical publisher-subscriber code, we conduct experiments under various conditions, including different message sizes, frequencies, node counts, and stress scenarios. Through this analysis, we demonstrate the advantages and trade-offs of using an RTOS like QNX in robotics and provide practical insights into the feasibility of adopting RTOS in real-time robot controllers.

2. EXPERIMENTAL ENVIRONMENTS

2.1 Target Operating Systems

To evaluate the real-time performance of ROS 2, we conducted experiments on three operating system environments: QNX 8.0 for hard real-time, Ubuntu 22.04 with the PREEMPT-RT kernel patch for soft real-time, and default Ubuntu 22.04 for non-real-time. These setups represent hard, soft, and non-real-time systems, respectively, enabling a direct comparison of their deterministic behavior under identical software conditions.

Note: QNX 8.0 was used under a non-commercial li-

This paper was supported by the BK21 FOUR Project.

cense and was employed solely for academic and experimental purposes.

2.2 Middleware Configurations

All systems were configured with ROS 2 Humble Hawksbill, the most widely adopted and actively maintained distribution at the time of the study. Its widespread use and long-term support make it a practical choice for robotic system development and performance evaluation. We adopted Fast DDS, the default middleware implementation in ROS 2 Humble, to ensure consistency across platforms and to reflect typical usage in real-world robotic applications.

2.3 Hardware Platforms

All experiments were executed on a Raspberry Pi 4 Model B equipped with 4 GB of RAM. This board was selected as a representative embedded platform commonly used in cost-sensitive and resource-constrained robotic systems. Compared to more powerful alternatives, for example, Intel NUC, the Raspberry Pi offers limited processing capabilities, making it ideal for evaluating performance under minimal hardware conditions.

This setup reflects realistic embedded deployment scenarios, where real-time robotic controllers must operate reliably despite limited computing resources.

3. MEASUREMENT SETUP

To evaluate the communication performance of ROS 2 across different operating systems, we implemented a set of tests using identical publisher-subscriber logic under a variety of conditions. All tests were conducted with the following QoS settings: reliability = reliable, history = keep last, and depth = 100.

```
void timer_callback()
{ // publisher callback
  if (count_ >= sample_count_) {
    rclcpp::shutdown();
    return;
  }
  msg.header.stamp = this->now();
  msg.header.frame_id = "msg_frame";
  publisher_>publish(msg);
  ++count_;
}

void topic_callback(const sensor_msgs::msg::Image::SharedPtr msg)
{ // subscriber callback
  received_msgs_.push_back(msg->header.stamp);
  received_times_.push_back(this->now());
}
```

Fig. 1. Example of Time stamp logging

To minimize any interference with real-time performance, both the publisher and subscriber callback functions were kept minimal. As shown in Figure 1, the publisher simply appends the current timestamp to the message, and the subscriber records its own reception time. No additional operations were performed within the callbacks to preserve accurate timing behavior.

The one-way latency L was calculated using the difference between the reception and transmission timestamps,

as follows:

$$L = t_{\text{recv}} - t_{\text{send}} \quad (1)$$

3.1 Message Structure and Node Configuration

All messages used in the experiments were official ROS 2 message types such as `sensor_msgs/Image` and `std_msgs/String`. These are categorized as *unplain* structures, as they include dynamically allocated memory through containers like `std::vector` or `std::string`. Although *plain* message structures with fixed size memory layout can benefit from shared memory (SHM) zero-copy transport, such messages are not commonly used in practical robotic systems.

To reflect realistic environments, we used the official ROS 2 unplain message `sensor_msgs/Image` throughout our experiments.

For the implementation, we adopted composable nodes[6] a commonly used deployment method in ROS 2 systems allowing multiple nodes to be instantiated in a single process. This enables intra-process communication with reduced latency and overhead, and is widely applied in production-level robotic systems.

Only the models in Figs. 2 and 4 were employed in our experiments.

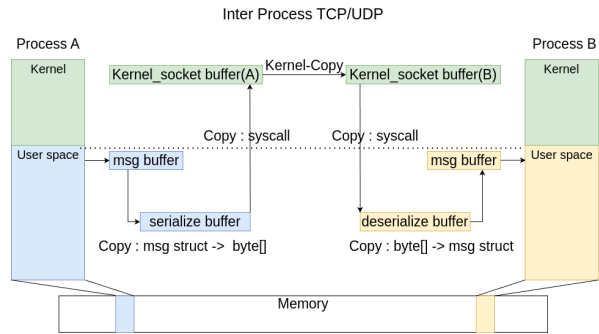


Fig. 2. Inter-process over TCP/UDP

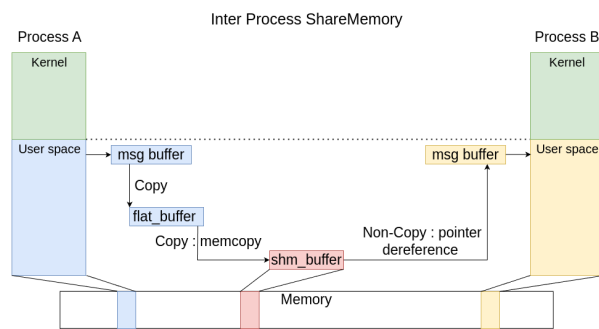


Fig. 3. Inter-process using shared memory

3.2 Test Scenarios

To comprehensively evaluate communication latency and real-time performance under realistic robotic workloads, we designed three major test scenarios:

1. **Frequency Test:** Varying message transmission frequencies from 10 Hz to 1000 Hz to evaluate how each operating system handles high-frequency communication.
2. **Payload Size Test:** Varying message sizes from 64 bytes to 16 megabytes to evaluate the impact of message

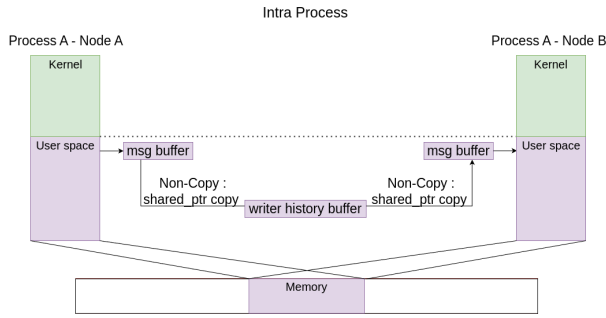


Fig. 4. Intra-process via composable nodes

volume on communication. Two distinct experiments were tested

- *Inter-process communication*: using standard pub-sub nodes across separate processes.
- *Intra-process communication*: using composable pub-sub nodes within the same process.

In both configurations, an **unplain message structure** was used, which reflects the common usage in ROS 2 where messages often include dynamically allocated fields. This setup makes a consistent and realistic evaluation of how payload affects IPC[7] performance in practical robotic deployments.

3. Stress and Determinism Test: To assess scheduling precision under high load, we introduced CPU-bound background processes and evaluated two conditions:

- Same-Priority Test*: All nodes run with equal priority, observing context switch latency and fairness.
- Priority-Based Test*: Explicit priority levels are assigned to each node, evaluating deterministic execution behavior, especially under QNX.

These scenarios were carefully selected to emulate typical deployment patterns of robotic systems and to stress the communication layers across both RTOS and GPOS platforms.

4. RESULT

This section presents the experimental results obtained under the test scenarios described earlier. In Section 4, we limit ourselves to reporting the findings, while detailed interpretation and conclusions are deferred to Chapter 5 (Discussion and Conclusion).

4.1 Frequency Variant Experiment

Figure 5 shows the mean end-to-end latency when transmitting and receiving a 1 KB sensor_msg payload at various publication frequencies. For both QNX and the Ubuntu PREEMPT_RT kernel, publisher and subscriber nodes were given high real-time priorities (90 and 250, respectively), excluding only core system interrupts and top level scheduler events. Among the entire frequency range from low to high QNX maintains substantially lower latency than Ubuntu_RT. Although the Ubuntu_RT patch reduces latency by a few milliseconds compared to the unpatched Ubuntu (vanilla) kernel, it remains outperformed by QNX on every measurement.

Table 1 summarizes the maximum latency and standard deviation for each OS under the same frequency sweep. In almost every test case, QNX achieves the lowest maximum latency and the tightest distribution (smallest standard deviation), followed by Ubuntu_RT and then vanilla Ubuntu. This ordering QNX, Ubuntu_RT, Ubuntu (vanilla) confirms that QNX delivers not only the lowest mean latency but also the most consistent, predictable performance.

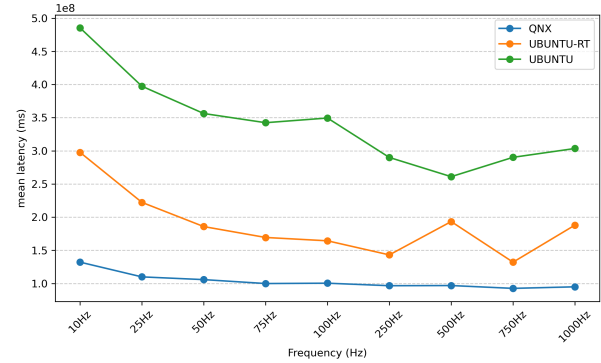


Fig. 5. Hz vs Latency Graph

Table 1. Max and Std of Frequency Experiment

hz	max_ub	max_ub-rt	max_qnx	std_ub	std_ub-rt	std_qnx
10	8.16E+08	4.73E+08	3.93E+08	9.39E+07	6.27E+07	3.75E+07
25	7.97E+08	1.18E+09	4.08E+08	7.87E+07	7.87E+07	2.56E+07
50	9.65E+08	4.93E+08	4.52E+08	7.13E+07	6.81E+07	2.20E+07
75	8.57E+08	5.15E+08	4.11E+08	6.04E+07	6.17E+07	1.86E+07
100	3.02E+09	4.48E+08	5.18E+08	1.51E+08	5.99E+07	2.08E+07
250	3.85E+09	5.19E+08	5.25E+08	1.74E+08	4.45E+07	1.71E+07
500	8.50E+09	8.33E+10	4.15E+08	1.48E+08	1.85E+09	1.07E+07
750	1.29E+10	5.38E+08	2.29E+08	2.69E+08	2.74E+07	7.92E+06
1000	1.47E+10	8.21E+10	2.78E+08	2.52E+08	1.81E+09	1.04E+07

4.2 DataSize Variant Experiment

For both QNX and the Ubuntu PREEMPT_RT kernel, publisher and subscriber nodes were configured with high real-time priorities (90 and 250, respectively) to ensure that scheduling interference from other system activities was minimized.

Figure 6 illustrates the average end-to-end latency measured in inter-process mode using a standard ROS 2 message (without shared memory). As payload size increases up to around 64 KB, all three platforms QNX, Ubuntu RT, and vanilla Ubuntu exhibit a steady rise in latency. In this regime, QNX consistently traces the lowest curve, suggesting that its kernel-level optimizations (such as more efficient user-kernel data transfers and buffer management) may be keeping message-passing overhead to a minimum. Beyond the 64 KB mark, however, QNX's latency climbs more sharply than the Linux-based systems. This behavior is likely driven by the compounded impact of packet fragmentation and reassembly, additional copy operations between user and kernel space, dynamic buffer allocation/deallocation, and increased context-switch frequency. In contrast, Ubuntu RT's more gradual latency increase for large payloads hints that its batch RX/TX processing and slab-allocator reuse are effectively amortizing per-packet overhead, while vanilla Ubuntu without hard-RT guarantees or these buffering

optimizations appears to accumulate the most latency at the largest message sizes.

Figure 7 repeats the data-size experiment in an intra-process configuration by deploying publisher and subscriber as composable nodes within the same process. This removes the inter-process communication layer and reduces serialization costs. As a result, all three systems achieve lower absolute latencies compared to the inter-process case, yet the overall trend remains: QNX leads for payloads up to 64 KB, while its performance advantage narrows and eventually reverses at larger sizes, where the Linux kernels' throughput-oriented optimizations begin to dominate.

Tables 2 and 3 present the full statistical breakdown maximum latency and standard deviation for both inter-process and intra-process modes. Across every data point, QNX attains the smallest peak values and the tightest timing distribution, followed by Ubuntu RT, with vanilla Ubuntu consistently trailing. These results confirm that QNX delivers the most deterministic performance for small to moderate message sizes, while the PREEMPT RT kernel offers a middle ground, and standard Ubuntu provides the least predictability under heavy payload loads.

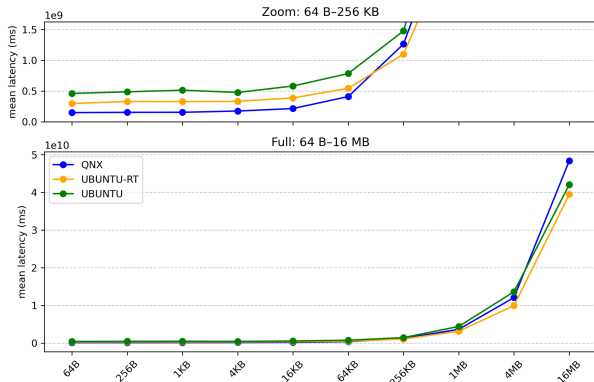


Fig. 6. Payload vs Latency Graph

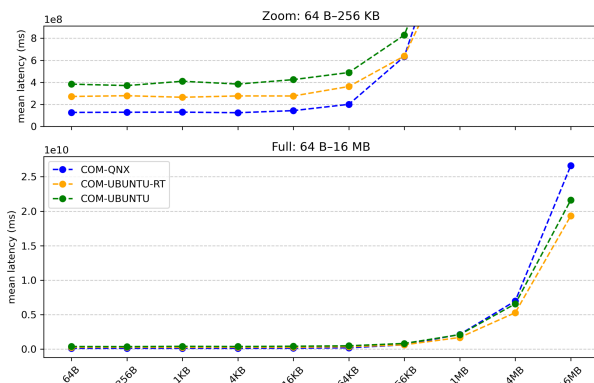


Fig. 7. Payload vs Latency Graph - Composable

4.3 Stress Condition Experiment

Figure 9 presents the mean round-trip latency over 60 s with a 1 KB payload at 100 Hz. To emulate a realistic ROS 2 control setup (15 to 30 nodes)[8] we ran a “stress” node that spawned 20 concurrent CPU-intensive

Table 2. Max and Std of Data Size Experiment

Byte	max_ub	max_ub-rt	max_qnx	std_ub	std_ub-rt	std_qnx
64 B	6.30E+05	4.65E+05	3.84E+05	7.67E+04	5.59E+04	4.39E+04
256 B	7.14E+05	5.75E+05	2.86E+05	8.34E+04	6.93E+04	4.19E+04
1 KB	9.44E+05	4.87E+05	3.73E+05	1.01E+05	5.68E+04	4.42E+04
4 KB	7.79E+05	4.67E+05	3.86E+05	8.62E+04	5.06E+04	4.95E+04
16 KB	1.06E+06	5.87E+05	5.35E+05	1.12E+05	5.70E+04	5.80E+04
64 KB	1.87E+06	1.12E+06	9.26E+05	1.37E+05	8.54E+04	9.13E+04
256 KB	3.84E+06	2.88E+06	1.74E+06	2.88E+05	1.91E+05	6.90E+04
1 MB	2.81E+07	1.99E+07	1.64E+07	2.44E+06	1.70E+06	1.29E+06
4 MB	4.20E+07	4.40E+07	2.89E+07	3.30E+06	3.55E+06	1.71E+06
16 MB	1.72E+08	1.59E+08	7.96E+07	1.78E+07	1.53E+07	3.44E+06

Table 3. Max and Std of Composable Node Experiment

Byte	max_ub	max_ub-rt	max_qnx	std_ub	std_ub-rt	std_qnx
64B	9.12E+05	1.07E+06	4.64E+05	8.64E+04	1.06E+05	5.21E+04
256B	9.11E+05	9.05E+05	4.48E+05	8.36E+04	9.90E+04	5.29E+04
1KB	1.47E+06	9.96E+05	6.23E+05	1.40E+05	9.65E+04	6.65E+04
4KB	1.37E+06	7.94E+05	4.62E+05	1.25E+05	9.35E+04	5.16E+04
16KB	1.47E+06	1.18E+06	6.70E+05	1.26E+05	1.23E+05	7.14E+04
64KB	1.75E+06	1.60E+06	7.59E+05	1.48E+05	1.45E+05	7.91E+04
256KB	3.02E+06	2.90E+06	1.82E+06	2.34E+05	2.38E+05	1.55E+05
1MB	7.69E+06	7.46E+06	4.96E+06	7.20E+05	6.12E+05	2.93E+05
4MB	2.39E+07	2.39E+07	1.51E+07	1.92E+06	2.01E+06	8.30E+05
16MB	9.18E+07	8.80E+07	5.35E+07	7.72E+06	7.52E+06	2.74E+06

tasks and then measured subscriber-publisher latency under that load.

As shown in Figure 8 all stress-node processes ran at the default priority. On native Ubuntu, no processes received real-time scheduling. On Ubuntu-RT and QNX, we compared two modes: “no-prior,” where subscriber and publisher also ran at default priority, and “prior,” where only those ROS 2 tasks were promoted to SCHED_FIFO. In the prior mode, the publisher and subscriber were assigned high real-time priorities 90 on Ubuntu-RT and 250 on QNX to minimize scheduling interference from other system activities. This isolates the effect of hard real-time priority on communication latency under heavy CPU contention.

Table 4 summarizes the maximum latency and standard deviation observed in this experiment. As expected, native Ubuntu exhibited the highest peak latency. Unexpectedly, QNX’s maximum latency and variance exceeded those of Ubuntu-RT in both the “no-prior” and “prior” configurations.

As a result, QNX was outperformed by Ubuntu-RT in both scheduling configurations, with higher mean latency, maximum latency, and standard deviation.

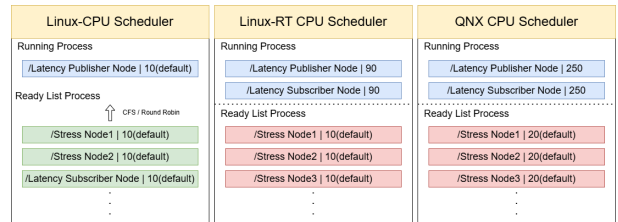


Fig. 8. Type of Scheduler

5. DISCUSSION AND CONCLUSION

5.1 Discussion

In the frequency-variation experiment, all operating systems exhibited higher latency at low frequencies (10–50 Hz) than at high frequencies (100–1000 Hz).

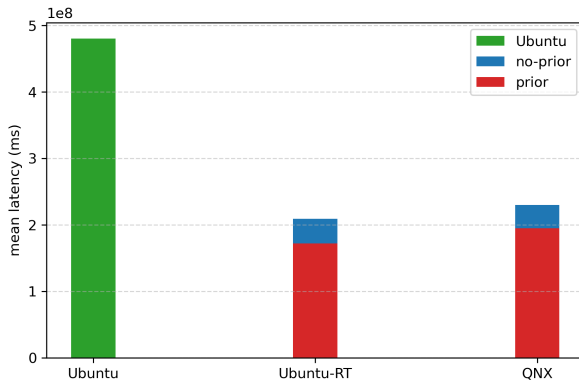


Fig. 9. Stress Condition Experiment

Table 4. Max and Std of Stress Experiment

Mode	Max (ms)	Std (ms)
Ubuntu (native)	4.81E+08	1.23E+06
Ubuntu-RT no-prior	2.09E+08	9.87E+05
Ubuntu-RT prior	1.72E+08	8.76E+05
QNX no-prior	2.30E+08	1.12E+06
QNX prior	1.95E+08	1.01E+06

We attribute this to the Raspberry Pi 4 down-clocking into a low-power mode under light communication loads, which limits CPU performance.[9] In practical robotics, 10–50 Hz control loops are common,[10] and NVIDIA Isaac Sim recommends 100–150 Hz for smooth motion; to explore edge cases, we extended tests up to 1000 Hz.

Assigning real-time priority to ROS 2 nodes improved latency even in background-free environments. As shown in Figure 10, high-priority nodes (SCHED_FIFO) preempt default-priority tasks like idle daemons and kernel services, reducing jitter and context switch delay. This indicates that even without external interference, default-priority tasks can introduce timing noise, which can be suppressed via priority-based scheduling. In short, native preemptive schedulers improve response latency.

Across all tests, QNX consistently delivered the lowest latency, confirming its real-time advantage under high-speed communication compared to general-purpose Linux. In the data-size variation experiment, QNX had the lowest latency for messages 64 KB, but degraded with larger sizes. We infer that QNX’s microkernel IPC incurs overhead when fragmentation and memory copies increase. A similar pattern appeared in the intra-process (Composable Node) mode, where QNX excelled for small messages but underperformed with large ones.

In the stress-condition experiment, vanilla Ubuntu (no real-time priority) showed the highest latency in a 20-node, multi-process setup. Surprisingly, QNX exhibited slightly higher latency than Ubuntu RT, regardless of real-time priority assignment. As shown in Table 4, QNX’s maximum latency and standard deviation exceeded those of Ubuntu RT. We infer that QNX services like networking and file I/O run in user space, competing with stress tasks and increasing context switches and IPC contention. Precise analysis is limited due to lack of kernel access in

the non-commercial QNX image.

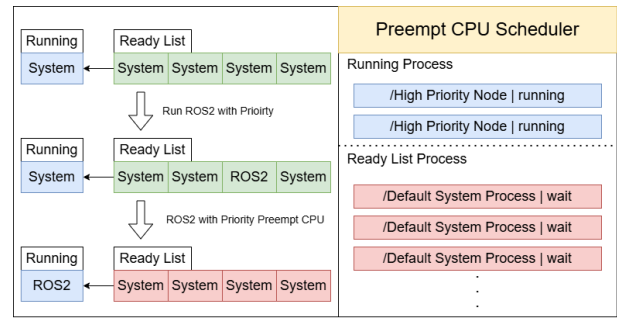


Fig. 10. Preemptive Scheduling Effect

5.2 Conclusion

In summary, our experiments demonstrate that real time operating systems such as QNX provide highly deterministic performance for ROS 2 based systems, particularly in high frequency small payload communication scenarios. This is evidenced by consistently lower standard deviations and maximum latencies. However, performance degradation was observed in complex multi node configurations commonly found in upper level control stacks due to increased context switching and scheduling contention.

As illustrated in Figure 11, uniformly assigning high real time priority to multiple ROS 2 nodes can lead to starvation[11] of lower priority processes, resulting in unbalanced and inefficient system behavior. These findings highlight that RTOS based scheduling must be carefully tuned in multi node scenarios. In particular, high priority tasks should be designed to periodically release the CPU (through explicit sleeping or yielding) to allow background or lower priority tasks to make progress. Without such cooperative mechanisms, fixed priority preemptive scheduling may cause long term unfairness, resource monopolization, and system instability.

To address these limitations, we propose a task separation architecture (Figure 12), where an RTOS (QNX) handles time critical low level operations such as motor control and sensor feedback, while a GPOS (Linux) executes compute intensive non real time workloads like path planning and SLAM. This layered architecture aligns with the modular design principles of ROS 2 and ensures that both real time determinism and computational scalability are preserved.

Furthermore, for single board deployments that require both functionalities, we suggest a hypervisor based system architecture (Figure 13). This approach isolates RTOS and GPOS environments on dedicated CPU cores, allowing each to operate independently while meeting timing constraints and performance demands.

Overall, our findings provide practical design guidance for robotic system developers using ROS 2, especially in scenarios requiring a careful balance between real time responsiveness and high level computation. By applying task level architectural separation, priority aware scheduling, and hybrid deployment models, developers can construct scalable, deterministic, and re-

liable robotic platforms suited for modern autonomous applications. Furthermore, this study provides concrete empirical evidence for the adoption of RTOS in robotic systems. The implementation code is available at https://github.com/wasabi9812/ros2_latency_measurement.

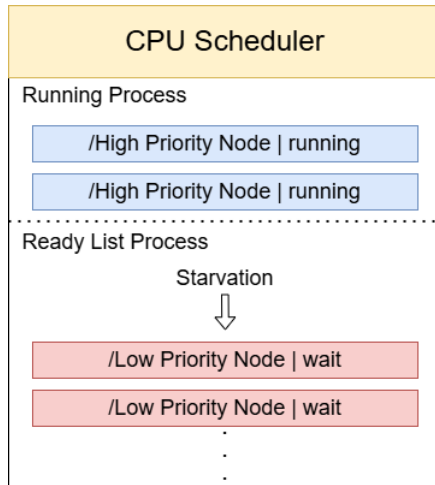


Fig. 11. Starvation Condition

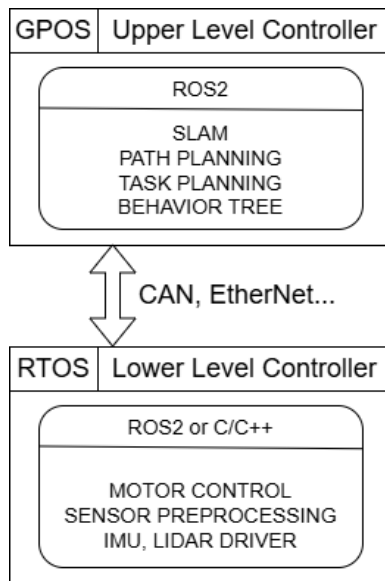


Fig. 12. Proposed upper-lower controller separation

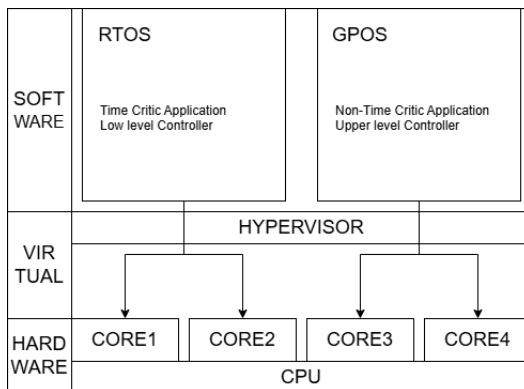


Fig. 13. Hypervisor-based architecture

REFERENCES

- [1] Nicolas Santi and Nathalie Mitton. A resource management survey for mission-critical and time-critical applications in multiaccess edge computing. *ITU Journal on Future and Evolving Technologies*, 2(2):419–433, November 2021.
- [2] Ingo Molnár. The completely fair scheduler. In *Proceedings of the Linux Symposium*, volume 2, pages 1–14. Linux Foundation, June 2007.
- [3] Alan Burns and Andy J. Wellings. The ravenstar profile for high integrity real-time systems. *Journal of Systems Architecture*, 46(6):387–407, June 2000.
- [4] S. Pareek and K. Kumar. Microkernel operating system architecture and qnx. In *2016 International Conference on Advances in Computing, Communications and Informatics (ICACCI)*, pages 123–128. IEEE, September 2016.
- [5] Tobias Kronauer, Joshwa Pohlmann, Maximilian Matthe, Till Smejkal, and Gerhard Fettweis. Latency analysis of ros 2 multi-node systems. *arXiv preprint arXiv:2101.02074*, 2021.
- [6] Steve Macenski, Alberto Soragna, Michael Carroll, and Zhenpeng Ge. Impact of ros 2 node composition in robotic systems. *arXiv preprint arXiv:2305.09933*, 2023.
- [7] Michael A. Accetta, Robert V. Baron, William J. Bolosky, David S. Golub, Richard F. Rashid, Arthur Tevanian, and Michael Young. Mach: A new kernel foundation for unix development. *USENIX Conference Proceedings*, pages 93–113, 1986.
- [8] Steve Macenski, Alberto Soragna, Michael Carroll, and Zhenpeng Ge. Impact of ros 2 node composition in robotic systems. *arXiv preprint arXiv:2305.09933*, 2023.
- [9] Y. Watanabe, T. Takaki, N. Fukumoto, and H. Kawashima. Real time capability test on the raspberry pi based epics controller. In *Proceedings of the 17th Annual Meeting of the Particle Accelerator Society of Japan (PASJ)*, pages 1127–1130, 2020.
- [10] Steve Macenski, Alberto Soragna, Michael Carroll, and Zhenpeng Ge. Impact of ros 2 node composition in robotic systems. *arXiv preprint arXiv:2305.09933*, 2023.
- [11] C. L. Liu and James W. Layland. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM (JACM)*, 20(1):46–61, 1973.